Q) Implement Min, Max, and Sum and Average operations using Parallel Reduction.

```
#include <iostream>      // For input and output (cout, cin)
#include <omp.h>         // For OpenMP parallel programming
#include <climits>       // For INT_MAX and INT_MIN constants

using namespace std;

// Function to find minimum value using parallel reduction
void min_reduction(int arr[], int n)
{
    int min_value = INT_MAX;   // Initialize min_value with largest possible
integer

    // OpenMP parallel loop with reduction (min operation)
    #pragma omp parallel for reduction(min: min_value)
    for (int i = 0; i < n; i++)
    {
        // Compare each element with current min_value
        if (arr[i] < min_value)
        {
            min_value = arr[i];   // Update min_value if smaller element found
        }
    }

    // Print minimum value
    cout << "Minimum value: " << min_value << endl;
}

// Function to find maximum value using parallel reduction
void max_reduction(int arr[], int n)
{
    int max_value = INT_MIN;   // Initialize max_value with smallest possible
integer

    // OpenMP parallel loop with reduction (max operation)
    #pragma omp parallel for reduction(max: max_value)
    for (int i = 0; i < n; i++)
    {
        // Compare each element with current max_value
        if (arr[i] > max_value)
        {
            max_value = arr[i];   // Update max_value if larger element found
        }
```

```
    }

    // Print maximum value
    cout << "Maximum value: " << max_value << endl;
}

// Function to calculate sum using parallel reduction
void sum_reduction(int arr[], int n)
{
    int sum = 0;   // Initialize sum to 0

    // OpenMP parallel loop with reduction (addition)
    #pragma omp parallel for reduction(+: sum)
    for (int i = 0; i < n; i++)
    {
        sum += arr[i];   // Add each element to sum
    }

    // Print total sum
    cout << "Sum: " << sum << endl;
}

// Function to calculate average using parallel reduction
void average_reduction(int arr[], int n)
{
    int sum = 0;   // Initialize sum to 0

    // OpenMP parallel loop with reduction (addition)
    #pragma omp parallel for reduction(+: sum)
    for (int i = 0; i < n; i++)
    {
        sum += arr[i];   // Add each element
    }

    // Calculate and print average
    // Note: should be sum/n, not (n-1)
    cout << "Average: " << (double)sum / n << endl;
}

// Main function
int main()
{
    int *arr, n;   // Pointer for array and variable for size

    // Take number of elements from user
```

```
    cout << "\nEnter total number of elements => ";
    cin >> n;

    arr = new int[n];   // Dynamically allocate array of size n

    // Take array elements as input
    cout << "\nEnter elements => ";
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

    // Call reduction functions
    min_reduction(arr, n);     // Find minimum value
    max_reduction(arr, n);     // Find maximum value
    sum_reduction(arr, n);     // Find sum
    average_reduction(arr, n); // Find average

    return 0;   // End of program
}
```

Output-

```
Enter total number of elements => 5

Enter elements => 1 3 5 6 8 9
Minimum value: 1
Maximum value: 8
Sum: 23
Average: 4.6
10


...Program finished with exit code 0
Press ENTER to exit console.
```